Singular Value Decomposition (SVD) Implementation Results

Rushil Gupta

Introduction

This document presents the results of implementing Singular Value Decomposition (SVD) using custom algorithms and compares them with NumPy's built-in SVD functionality. The primary focus is on performance metrics and accuracy of the decomposition.

Methodology

The custom SVD implementation utilizes Householder reflections for QR factorization and the QR algorithm for eigen-decomposition of A^TA. The overall algorithm follows these steps:

1. Compute A^TA.
2. Apply the QR algorithm to find the eigenvalues and eigenvectors of A^TA.
3. This gives us $A^TA = V\Lambda V^T$.
4. Sort the eigenvalues in descending order to obtain the singular values.
5. By definition, $\Sigma = \sqrt{\Lambda}$, and $U\Sigma = AV$.
6. Then, it is straightforawrd to get U since Σ is diagonal.

NumPy's optimized `numpy.linalg.svd` function is used as a benchmark for comparison in time. Another error metric is the reconstruction error $\|A - U\Sigma V^T\|$.

Results

Performance Comparison

The table below summarizes the average runtime of the custom SVD implementation versus NumPy's SVD over multiple trials:

Accuracy Comparison

The reconstruction error $\|A - U\Sigma V^T\|$ for the method is presented below:

Table 1: Average Runtime (seconds)

Matrix Size	Custom SVD (s)	NumPy SVD (s)
50×50	9.4×10^{-2}	5.2×10^{-4}
100×100	2.9×10^{-1}	4.7×10^{-3}
500×500	1.62×10^{1}	6.7×10^{-3}
1000×1000	$.57 \times 10^{2}$	8.1×10^{-3}

Table 2: Reconstruction Error

Matrix Size	Custom SVD Error
50×50	1.08×10^{-13}
100×100	2.70×10^{-13}
500×500	3.00×10^{-12}
1000×1000	9.78×10^{-12}

Conclusion

The custom SVD implementation successfully decomposes matrices and reconstructs them with reasonable accuracy.